

UNIT 3, AGENT DEVELOPMENT WITH PYTHON AND FRAMEWORKS

LECTURE 4

Memory, State, and Retrieval

Your agent can act, but it forgets, and it makes things up.

Today, two real fixes: a session that remembers across turns,
and retrieval that lets it answer from your documents, not its guesses.

SECTION 00

Two problems left

It forgets between turns, and it invents facts. Fix both.

What your agent still cannot do

PROBLEM ONE

It forgets

Each run starts fresh. Ask a follow up and it has no idea what you were talking about. You built a Session by hand in Unit 2 to fix this. Now the framework gives you one.

PROBLEM TWO

It invents

Ask it about your company policy and it answers confidently from its training, which never saw your policy. It does not know what it does not know. This is the dangerous one.

TWO FIXES, BOTH REAL

Memory fixes the forgetting: a session that carries the conversation. Retrieval fixes the inventing: search your own documents and put the facts in front of the model before it answers. The second one, retrieval augmented generation, is one of the most valuable things in this whole course.

Today

- Sessions: memory across turns, given to you by the framework.
- Why a model inventing facts is a structural problem, not a bug you can prompt away.
- Retrieval augmented generation: the shape that fixes it, built from scratch.
- Chunking, embedding, and cosine similarity, the three moving parts of search.
- The honest limits of retrieval, and why real systems use semantic embeddings.

HOLD ON TO THIS

You will build a working retrieval store today, small enough to read in one sitting, and see it pull the right paragraph out of a pile of documents. Then you hand that paragraph to the agent and watch it answer from your facts instead of its guesses.

SECTION 01

Memory, the easy half

The framework hands you the session you built by hand.

A session, given not built

```
session = agent.create_session()

await agent.run("Route ticket 12 to billing", session=session)
await agent.run("Actually make it urgent", session=session)
#   ^ the second call KNOWS about the first, because
#     they share a session. "it" resolves to the ticket.
```

THIS IS YOUR UNIT 2 SESSION, OWNED BY THE FRAMEWORK

In Unit 2 you built a Session class: a bounded transcript plus pinned facts, and you threaded it through every turn by hand. The framework now does exactly that. You create a session, pass it to each run, and the conversation carries. Same idea, same behaviour, none of the plumbing. And because you built the plumbing once, you know precisely what it is doing for you.

Memory is solved, so move on

WHAT YOU GET FREE

The mechanics

Creating the session, storing turns, threading it through runs, trimming it when it grows. All the manual work from Unit 2, handled.

WHAT IS STILL YOURS

The judgement

What to pin, when to start a fresh session, how much history matters. The framework gives you the tool. Deciding how to use it is still an engineering choice you own.

HOLD ON TO THIS

This is the pattern of the whole unit again: the framework takes the mechanics, you keep the judgement. Memory was hard to build and is easy to use. Retrieval, the rest of today, is where the real thinking lives.

SECTION 02

The inventing problem

Why a model makes things up, and why prompting cannot fix it.

Why the model invents

The model answers from its training. Your documents were never in its training.

Ask about your company refund policy and the model produces a fluent, plausible, confident answer, assembled from ten thousand other companies' policies it did see. It is not lying. It genuinely does not know it is missing your specific facts. It cannot tell the difference between knowing and guessing.

THIS IS STRUCTURAL, NOT A BUG

You cannot fix this by telling the model "do not make things up". It has no way to know which of its answers are grounded and which are invented, because to the model they feel identical. The only real fix is to stop asking it to remember, and start giving it the facts to read. That is retrieval.

The idea, in one line

Do not ask the model to KNOW your facts. Retrieve them, and ask it to READ.

Retrieval augmented generation. Before the model answers, you search your own documents for the pieces relevant to the question, and you put those pieces in the prompt. The model then answers from what is in front of it, not from what it half remembers. You changed the question from "what do you know" to "what does this say".

THIS IS THE FOUNDRY KNOWLEDGE PANEL, FOR REAL

Remember the Knowledge panel on the Foundry tour, the one I said we would build for real in Unit 5? This is it, one unit early and from scratch, so you understand exactly what it does before you ever click it.

SECTION 03

Building retrieval

Three moving parts: chunk, embed, search.

Part one, chunking

```
def chunk_text(text, max_words=40):  
    words = text.split()  
    return [" ".join(words[i:i+max_words])  
            for i in range(0, len(words), max_words)]
```

WHY NOT JUST EMBED THE WHOLE DOCUMENT

If you turn a ten page document into one vector, a search matches the document as a blurry average, and you hand the model ten pages to find one sentence in. Chunking into small pieces lets you retrieve the exact paragraph that answers the question. Precise retrieval beats a big pile of maybe-relevant text, every time.

Part two, embedding

THE LAYMAN VERSION

Turn meaning into numbers

An embedding is a list of numbers that captures what a piece of text means. Texts about similar things get similar numbers. It is a location in a space where nearby means related.

THE TECHNICAL VERSION

A vector in semantic space

A model maps text to a fixed length vector so that semantic similarity becomes geometric closeness. "refund" and "money back" land near each other even though they share no words.

THE ONE THING TO HOLD ONTO

Embedding converts the fuzzy human question "what does this mean" into the precise machine question "how close are these two vectors". Once meaning is numbers, finding related text becomes arithmetic. That conversion is the whole trick behind semantic search.

Part three, search by similarity

```
def search(query, top_k=3):  
    q = embed(query)                # the question, as a vector  
    scored = [(chunk, cosine(q, chunk.vector))  
               for chunk in self.chunks] # score every chunk  
    scored.sort(reverse=True)         # closest first  
    return scored[:top_k]            # keep the best few
```

COSINE SIMILARITY IS THE WHOLE ENGINE

Embed the query, measure how close it is to every stored chunk with one number, sort, take the best. That is semantic search, top to bottom. And notice: no model runs during the search itself. The model only comes in afterwards, to write an answer from the chunks you found. Retrieval is arithmetic; generation is the model.

Putting it together, the RAG shape

1 RETRIEVE

Search your documents for chunks relevant to the question



2 ASSEMBLE

Put those chunks into the prompt as context



3 INSTRUCT

Tell the model: answer ONLY from this context



4 GENERATE

The model writes an answer grounded in your facts

HOLD ON TO THIS

Steps one to three are yours, ordinary code, no model. Step four is the single model call, and by then it has your facts in hand. You spent almost nothing to make the model far more truthful.

SECTION 04

The honest limits

Where retrieval breaks, and what real systems do.

The toy embedder has a real weakness

WHAT WE BUILT

Word matching

Our teaching embedder matches on shared words. Search "refund" and it finds chunks containing "refund". Simple, offline, and enough to see the mechanics.

WHERE IT BREAKS

Meaning without shared words

Search "refund" and it misses a chunk that says "money back", because they share no words. It also misses "refunds", the plural. Word matching is not meaning matching.

THIS IS EXACTLY WHY REAL EMBEDDINGS EXIST

A real embedding model puts "refund", "refunds" and "money back" near each other in vector space, because it captures meaning, not spelling. Swap our toy embedder for a real one and the same RagStore does true semantic search, no other change. That swap is a single line, because we hid the embedder behind a seam, the same trick as the Transport in Unit 2 Lecture 3.

When retrieval itself fails

NOTHING RELEVANT

The answer is not in your documents at all. A good RAG system says "I do not have that", it does not fall back to inventing. The instruction to use only the context is what enforces this.

THE WRONG CHUNK

Retrieval pulls a plausible but wrong passage. The model answers confidently from it. Garbage in, confident garbage out. Retrieval quality is answer quality.

TOO MUCH CONTEXT

You retrieve twenty chunks to be safe, bury the answer in noise, and pay for every token. Precise beats plentiful, which is why chunking and top_k matter.

HOLD ON TO THIS

RAG does not make hallucination impossible. It makes it controllable, if your retrieval is good and your instruction is strict. The reliability skills from Unit 2 apply here just as hard: test it, and do not trust it blindly.

Hold on to this

Do not ask the model to know your facts. Retrieve them, and ask it to read.

Sessions give memory

Your Unit 2 Session, owned by the framework. Create, pass, done.

Inventing is structural

The model cannot tell knowing from guessing. Prompting cannot fix it.

Retrieval is arithmetic

Chunk, embed, cosine search. No model until the final answer.

Retrieval quality is answer quality

A wrong chunk gives a confident wrong answer. Test it.

Before next session

DO

Build a retrieval store

Load three or four short documents of your own into the RagStore. Ask questions and watch which chunks come back. Find a question it retrieves well and one it retrieves badly.

DO

Break it on purpose

Ask the store something that is NOT in any document. Confirm your RAG agent says it does not know, rather than inventing. If it invents, tighten the instruction until it stops.

THINK

Word versus meaning

Find two phrases that mean the same thing but share no words. Confirm the toy embedder misses the link. Write one sentence on why a real embedding would catch it.

ON THE SECOND ONE

Getting your agent to admit it does not know is harder and more valuable than getting it to answer. An agent that says "that is not in my documents" is trustworthy. One that invents an answer is dangerous. The instruction to answer only from context is what separates them, so test it.

Next session

Unit 3, Lecture 5

Multi-Agent Systems, the Graph Model

You saw the door in Lecture 3: an agent can be another agent's tool. Next time we walk through it. One agent grows into several that pass work between them, and the conversation-first model of the old AutoGen gives way to the graph-based model of Agent Framework, the harder, more powerful shift that closes the unit and sets up Unit 4.

COME WITH

Your retrieval store loaded, your does-not-know test passing, and two same-meaning phrases the toy embedder misses.